

Evolving Neural Networks through Augmenting Topologies

AVINASH CHAUDHARI AND YASH SABALE

April 2025

§1 Problem Statement

We aim to implement the NeuroEvolution of Augmenting Topologies (NEAT) algorithm from the seminal paper by Stanley & Miikkulainen (2002) to train neural networks for autonomous car driving in a 2D simulation environment. The key challenges include:

- Implementing NEAT’s core features: historical markings, speciation, and incremental topology growth
- Designing a fitness function that rewards track following and speed control
- Integrating Pygame-based car physics with NEAT’s genetic algorithm components

§2 Implementation Details

§2.1 NEAT Core Components

- **Genetic Encoding:** Uses innovation numbers (e.g., `ConnectionGene.innov`) to track historical origins of nodes/connections[7]
- **Speciation:** Implements compatibility distance calculation using disjoint/excess genes and weight differences
- **Mutation:** Implements add-node (splitting connections) and add-connection mutations

§2.2 Car Environment

- Simulating Car motion using a custom Physics engine
- Track editor with mouse-driven waypoint creation and right-click undo
- Collision detection using rotated car rects and track line segments

§2.3 Network Architecture

- Uses PyTorch for neural network implementation with custom activation functions (sigmoid, ReLU, tanh)
- Fitness calculation combines:

$$\text{Fitness} = 0.7 \times \text{distance_traveled} + 0.3 \times \text{speed_maintained} - 0.2 \times \text{collisions}$$

§3 Current Progress

Component	Status
Car physics simulation	Complete
Basic NEAT structure	Partial
Track generation system	Functional
Speciation mechanism	In progress
Fitness evaluation	Not implemented

Link to the current code: [drive link](#)

§4 Timeline

- **Week 1 (Apr 15-21):**
 - Complete speciation and fitness calculation
 - Implement population initialization with minimal topologies
 - Add sensor inputs (distance to track edges)
- **Week 2 (Apr 22-28):**
 - Implement crossover with historical markings
 - Add recurrent connection support
 - Optimize network forward propagation[5]
- **Final Week (Apr 29-May 2):**
 - Tune mutation rates and speciation thresholds
 - Conduct hyperparameter optimization
 - Prepare demonstration with visualization

§5 Potential Improvements: Hybrid NEAT-Backpropagation Approach

§5.1 Integration Strategy

- **Hybrid Training Loop:** Alternate between NEAT's structural mutations and backpropagation weight optimization phases

§5.2 Architecture Modifications

Convert `NN.forward()` to use PyTorch's autograd-compatible operations

§5.3 Implementation Challenges

Challenge	Solution
Dynamic computation graphs	Creating new computational graph for each generation
Fitness differentiability	Using Deep RL techniques

§5.4 Expected Benefits

- Hopefully faster weight convergence compared to mutation-only approach
- Better preservation of useful structure through gradient-aware crossover
- Automatic learning rate adaptation via Adam optimizer integration

This hybrid approach leverages PyTorch’s automatic differentiation capabilities while maintaining NEAT’s topological exploration strengths. The existing `bias` and `weight` parameters can be directly optimized using standard backpropagation when paired with appropriate loss formulations.

§6 References

1. Stanley, K.O., & Miikkulainen, R. (2002). Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2):99-127[7]
2. Pygame Documentation. (2025). Car simulation framework
3. PyTorch Documentation. (2025). Neural network implementation